

Міністерство освіти і науки України
Національний університет "Львівська політехніка"



Звіт до лабораторної роботи №8
З дисципліни "Web-технології та Web-дизайн"
на тему: "React.js: Item page"

Виконала:

Ст.гр. ІР-11

Луцан І.З.

Прийняв:

доцент, к.т.н.

Іванюк О.О.

Львів 2025

Завдання:

8. React.js: Item page

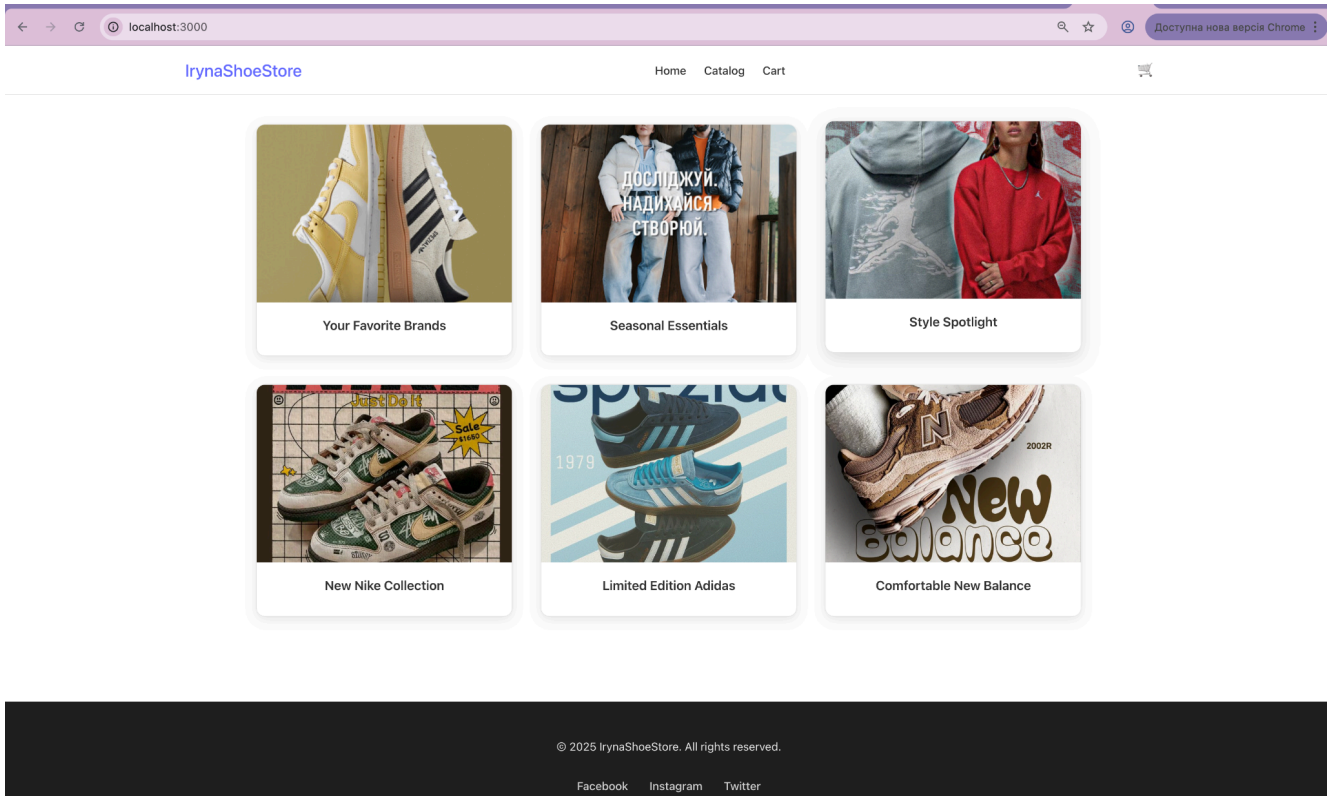
Description: Continue work on your React App by adding a page for your Item (see the link to wireframe of **Item page** above). Also, now, you have to make all your previous pages (Home & Catalog) **more interactive**.

Variants - (products that you are 'selling') the same as for previous works. (see the description to 3rd work)

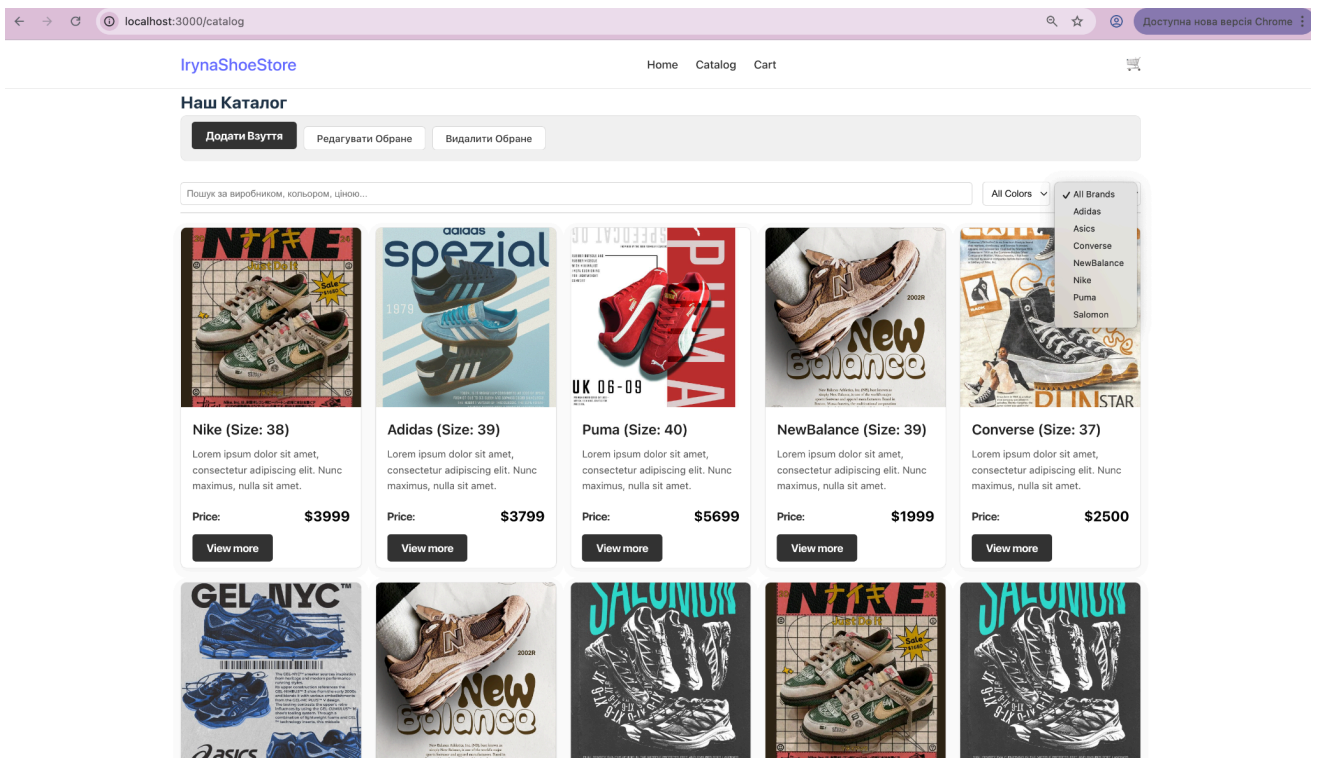
Requirements:

- All of the requirements for previous React.js works should be kept.
- **Code style:**
 - Your **items** should be stored inside the **state or context (your choice)** of your page
<https://uk.reactjs.org/docs/hooks-state.html>
<https://uk.reactjs.org/docs/hooks-reference.html#usecontext>
 - For your state management use **useState()** inside **Functional Component** instead of **this.state** and **Class component**
 - If you decided to use context, use **useContext()** hook instead of **Context.Consumer**
<https://www.robinwieruch.de/react-usecontext-hook>
- **Functionality (IMPORTANT):**
 - **Home page:** "View more" button should display more elements on the same page *Tip:* Elements can be just random paragraph & heading, use your imagination ;)
 - **Catalog page:** You should be able to filter your items list, by applying different filters by item's properties (i.e size/color/type)
 - **Catalog page:** Search by any text property option should also work
 - **Catalog & Item pages:** "View more" action on every item should refer to corresponding **Item page**, with correct information about item (get the info from your **state/context**)

Вигляд Home page коли натискаєш кнопку view more:



Оновлена Catalog page у якій вже є працюючі фільтри та вікно пошуку. Кнопки Додати, Редагувати, Видалити також працюючі



Оновлений NewsList.jsx

```
import { useState, useEffect } from 'react';
import ProductCard from '../ProductCard/ProductCard.jsx';
import './NewsList.css';

function NewsList() {
  const [featuredNews, setFeaturedNews] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [visibleCount, setVisibleCount] = useState(3);

  useEffect(() => {
    const fetchFeaturedNews = async () => {
      try {
        const response = await fetch('http://localhost:5001/api/featured-news');

        if (!response.ok) {
          throw new Error('Failed to fetch featured news');
        }

        const data = await response.json();
        setFeaturedNews(data);
      } catch (err) {
        setError(err.message);
        console.error('Error fetching featured news:', err);
      } finally {
        setLoading(false);
      }
    };

    fetchFeaturedNews();
  }, []);

  // логіка для кнопки "View More"
  const handleViewMore = () => {
    const step = 3;
    const totalItems = featuredNews.length;

    // оновлюємо стан збільшуючи на 3, але обмежуємо загальною кількістю
    setVisibleCount(prevCount => Math.min(prevCount + step, totalItems));
  };

  if (loading) {
    return (
      <section className="news-list">
        <div className="container">
          <h2 className="section-title">New Arrivals</h2>

```


Файл CatalogPage.jsx:

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import ShoeCard from '../containers/Components/ShoeCard/ShoeCard';
import SecondaryButton from '../containers/Components/ui/SecondaryButton';
import PrimaryButton from '../containers/Components/ui/PrimaryButton';
import { Link } from 'react-router-dom';

const gridStyles = {
  display: 'grid',
  gridTemplateColumns: 'repeat(auto-fill, minmax(250px, 1fr))',
  gap: '20px',
  padding: '20px 0'
};

const filterBarStyles = {
  display: 'flex',
  flexWrap: 'wrap',
  gap: '15px',
  alignItems: 'center',
  padding: '10px 0'
};

const adminPanelStyles = {
  padding: '15px',
  backgroundColor: '#f4f4f4',
  border: '1px solid #ddd',
  borderRadius: '8px',
  display: 'flex',
  gap: '10px',
  marginBottom: '20px'
};

function CatalogPage() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [selectedId, setSelectedId] = useState(null);
  const [isEditDisabled, setIsEditDisabled] = useState(true);
  const [isDeleteDisabled, setIsDeleteDisabled] = useState(true);
  const [searchTerm, setSearchTerm] = useState('');
  const [selectedColor, setSelectedColor] = useState('All');
  const [selectedProducer, setSelectedProducer] = useState('All');

  useEffect(() => {
    setIsEditDisabled(selectedId === null);
    setIsDeleteDisabled(selectedId === null);
  }, [selectedId]);

  // Обробник зміни фільтра кольору
  const handleColorChange = (e) => {
```

```

    setSelectedColor(e.target.value);
};

const handleProducerChange = (e) => {
    setSelectedProducer(e.target.value);
};

const fetchProducts = async () => {
    try {
        setLoading(true);
        setError(null);
        const response = await axios.get('/api/shoes');
        setProducts(response.data);
    } catch (err) {
        setError(err.message);
        console.error("Помилка завантаження взуття:", err);
    } finally {
        setLoading(false);
    }
};

useEffect(() => {
    fetchProducts();
}, []);

const handleDelete = async () => {
    if (!selectedId) {
        alert("Будь ласка, оберіть товар для видалення.");
        return;
    }

    if (window.confirm('Ви впевнені, що хочете видалити цей товар?')) {
        try {
            await axios.delete(`/api/shoes/${selectedId}`);
            setSelectedId(null);
            alert('Товар успішно видалено!');
            fetchProducts();

        } catch (err) {
            console.error("Помилка видалення:", err);
            alert("Не вдалося видалити товар: " + err.message);
        }
    }
};

// Отримуємо унікальні кольори та бренди
const allColors = [...new Set(products.map(p => p.color))].sort();
const allProducers = [...new Set(products.map(p => p.producer))].sort();

const filteredProducts = products.filter(product => {

```

```

    const colorFilterPassed = selectedColor === 'All' || product.color === selectedColor;
    const producerFilterPassed = selectedProducer === 'All' || product.producer ===
selectedProducer;

    if (!(colorFilterPassed && producerFilterPassed)) return false;

    const query = searchTerm.toLowerCase().trim();
    if (query === '') return true;

    // Логіка текстового пошуку
    const producerMatch = product.producer.toLowerCase().includes(query);
    const colorMatch = product.color.toLowerCase().includes(query);
    const priceMatch = String(product.price).includes(query);
    const sizeMatch = String(product.size).includes(query);

    return producerMatch || colorMatch || priceMatch || sizeMatch;
  });

  if (loading) return <p>Завантаження товарів...</p>;
  if (error) return <p>Сталася помилка: {error}</p>;

  return (
    <div>
      <h2>Наш Каталог</h2>

      <div style={adminPanelStyles}>
        <Link to="/catalog/new" style={{ textDecoration: 'none' }}>
          <PrimaryButton as="span">
            Додати Ввуття
          </PrimaryButton>
        </Link>

        <Link
          to={` /catalog/edit/${selectedId}`}
          style={{ textDecoration: 'none' }}
        >
          <SecondaryButton as="span" disabled={isEditDisabled}>
            Редагувати Обране
          </SecondaryButton>
        </Link>

        <SecondaryButton
          disabled={isDeleteDisabled}
          style={!isDeleteDisabled ? {backgroundColor: '#ffdddd', borderColor: '#ff5555',
color: '#d00'} : {}}
          onClick={handleDelete}
        >
          Видалити Обране

```



```

    </SecondaryButton>
  </div>

  <div style={filterBarStyles}>
    <input
      type="text"
      placeholder="Пошук за виробником, кольором, ціною..."
      value={searchTerm}
      onChange={(e) => setSearchTerm(e.target.value)}
      style={{ padding: '8px', border: '1px solid #ccc', borderRadius: '4px', flexGrow:
1 }}
    />

    <select
      value={selectedColor}
      onChange={handleColorChange}
      style={{ padding: '8px', border: '1px solid #ccc', borderRadius: '4px' }}
    >
      <option value="All">All Colors</option>
      {allColors.map(color => (
        <option key={color} value={color}>{color}</option>
      ))}
    </select>

    <select
      value={selectedProducer}
      onChange={handleProducerChange}
      style={{ padding: '8px', border: '1px solid #ccc', borderRadius: '4px' }}
    >
      <option value="All">All Brands</option>
      {allProducers.map(producer => (
        <option key={producer} value={producer}>{producer}</option>
      ))}
    </select>

  </div>
  <hr />

  <div style={gridStyles}>
    {filteredProducts.map(product => (
      <ShoeCard
        key={product.id}
        product={product}
        onSelect={() => {
          setSelectedId(prevId => prevId === product.id ? null : product.id);
        }}
        isSelected={product.id === selectedId}
      />
    ))}

    {filteredProducts.length === 0 && <p>За вашим запитом нічого не знайдено.</p>}
```

```
    </div>
  </div>
);
}

export default CatalogPage;
```

Посилання на лабораторну роботу на гітхабі:

https://github.com/IrynaLutsan/Web_Programming_Labs/tree/main/Labs_React

Висновок:

Ця лабораторна робота успішно перетворила React-додаток на повноцінний інтерактивний інструмент, реалізувавши повний цикл CRUD (Create, Read, Update, Delete) для товарів.

Основні досягнення:

- Динамічна Маршрутизація: Впроваджено маршрути з параметрами (/catalog/edit/:id) та хук useParams() для керування режимами редагування та перегляду деталей.
- Керування Інтерактивністю: Хук useState() дозволив реалізувати розширені функції каталогу: текстовий пошук, багатофакторну фільтрацію (за кольором та брендом) та функцію "View More" на головній сторінці.
- Робота з Ефектами: Використання useEffect() забезпечило коректне завантаження даних існуючого товару для форми редагування та обробку асинхронних операцій з API (запити POST/PUT/DELETE).